SIMATS ENGINEERING

ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO4**	**Assessment:**	**ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK**
Weightage:	**30%**	**Total Marks:**	**25**
CO4 Statement:	*Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.*		
Student Name:	**ANUSHYA DEVI V**	**Reg.No.:**	**192424013**
Department:		**Date:**	

ANNEXURE A
INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization
   A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

2. Cloud Server Memory Bottleneck
   A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

3. Autonomous Vehicle Data Processing
   An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

4. AI Inference Edge Device
   An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

5. High-Performance Gaming Console
   A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

***Code of Conduct**:*

I ANUSHYA DEVI V (192424013) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

1. SMARTPHONE PERFORMANCE OPTIMIZATION

Problem:

The smartphone experiences lag while switching between apps because the processor frequently waits for data and instructions from slower memory.

Role of Memory Hierarchy:

Memory	Role	Speed	Typical Use
L1 Cache	Stores frequently used instructions/data	Fastest	Current CPU operations
L2 Cache	Larger backup cache for L1	Very fast	Recently used data
L3 Cache	Shared cache between CPU cores	Slower than L1/L2	Common data between cores
DRAM	Main memory/RAM	Much slower	Running applications and OS

Proposed Hierarchical Redesign:

CPU → L1 Cache → L2 Cache → L3 Cache → DRAM

1. Increase L1 cache efficiency – Keep frequently accessed app instructions and data close to each CPU core.
2. Optimize L2 cache – Provide sufficient capacity for active application data to reduce DRAM accesses.
3. Use a larger shared L3 cache – Store common data used by multiple CPU cores and applications.
4. Use high-speed LPDDR5/LPDDR5X DRAM – Provides higher bandwidth and lower power consumption.
5. Improve cache management – Use intelligent prefetching and replacement policies to predict which app data will be needed next.
6. Reduce unnecessary DRAM access – Keep frequently used application data in L1/L2/L3 caches.

Expected Result:

The redesigned hierarchy reduces memory access latency, decreases DRAM traffic, improves CPU utilization, and makes app switching faster and smoother while also improving overall smartphone throughput.

Conclusion:

A balanced cache hierarchy with efficient L1, L2, L3 caches and high-speed DRAM can significantly reduce smartphone lag. The main goal is to keep frequently accessed data as close to the CPU as possible.

2. CLOUD SERVER MEMORY BOTTLENECK

Problem:

A cloud service provider experiences high latency in database queries because the CPU frequently waits for data from slower memory or storage.

Cache Hierarchy vs. Virtual Memory Paging

Feature	**Cache Hierarchy**	**Virtual Memory Paging**
Purpose	Speeds up CPU data access	Extends memory using storage
Levels	L1, L2, L3 cache	RAM + SSD/HDD
Speed	Very fast	Much slower when page faults occur
Managed by	Hardware/cache controller	Operating system/MMU
Best for	Frequently accessed database data	Data that cannot fit in RAM
Main problem	Cache misses	Page faults and disk I/O

Cache hierarchy:

CPU → **L1 → L2 → L3 → DRAM**

Frequently accessed database indexes and query data should remain in cache as much as possible.

Virtual memory paging:

If the required data is not available in RAM, the operating system moves pages between DRAM and SSD/storage. Excessive paging creates page faults and significantly increases query latency.

Recommended Improvements

1. Increase L3 cache capacity to store frequently accessed database data.
2. Use high-speed DDR5/LPDDR5 DRAM with sufficient capacity to keep active databases in RAM.
3. Use NVMe SSDs instead of traditional HDDs for faster paging and database storage.
4. Optimize database caching so frequently accessed indexes and records remain in RAM/cache.
5. Reduce page faults by allocating enough physical RAM for the database workload.
6. Use memory-aware query optimization to improve cache locality and reduce unnecessary memory accesses.
7. Use NUMA-aware memory allocation in large multi-socket cloud servers to reduce remote-memory access latency.

Recommendation

The provider should prioritize cache and DRAM improvements before relying on virtual-memory paging. A suitable hierarchy is:

CPU → L1/L2/L3 Cache → High-speed DDR5 DRAM → NVMe SSD

This keeps frequently accessed database data in fast memory, reduces page faults and storage I/O, and therefore reduces database query latency and improves cloud-server throughput.

3. AUTONOMOUS VEHICLE DATA PROCESSING

Problem:

An autonomous vehicle receives huge amounts of real-time data from cameras, LiDAR, radar, GPS, and other sensors. This data must be processed with very low latency and high bandwidth to make immediate driving decisions.

Evaluation of Memory Technologies

Memory	Speed	Capacity	Power	Best Use
SRAM	Very high	Small	Higher	CPU/GPU cache and critical sensor data
DRAM	High	Large	Moderate	AI models, sensor buffers and working data
MRAM	High	Medium	Low/non-volatile	Fast storage of critical data and system states

Proposed Memory Hierarchy

Sensors → SRAM Cache → DRAM → MRAM

1. **SRAM – Fastest Level**
    - Store frequently accessed sensor data and real-time processing instructions.
    - Place it close to CPU/GPU/AI accelerators.
    - Provides the lowest latency.
2. **DRAM – Main Working Memory**
    - Store large camera frames, LiDAR point clouds, radar data, and AI model parameters.
    - Use high-bandwidth DDR5/LPDDR5X or automotive-grade DRAM.
3. **MRAM – Fast Non-Volatile Memory**
    - Store critical system information, frequently used parameters, and important vehicle-state data.
    - Retains data even when power is removed.
    - Provides faster access and lower standby power than conventional storage.

Memory Flow

Sensor Data → SRAM → DRAM → AI Processing

Critical Persistent Data → MRAM

Recommended Design

- Use large, high-speed SRAM caches for time-critical operations.
- Use high-bandwidth DRAM as the main memory for large sensor datasets.
- Use MRAM for low-power, non-volatile storage of critical data.
- Apply prefetching and intelligent cache management to reduce memory stalls.
- Use multiple DRAM channels to increase parallel data transfer.

Conclusion

The optimized hierarchy should use SRAM for minimum latency, DRAM for high-capacity/high-bandwidth processing, and MRAM for fast, low-power non-volatile storage.

Result:

This design enables faster sensor processing, reduces latency, maximizes memory bandwidth, and helps the autonomous vehicle make real-time driving decisions safely and efficiently.

4. AI INFERENCE EDGE DEVICE

Problem:

An edge AI device needs to load AI models quickly while consuming very little power. The memory system must provide a good balance between speed, capacity, and energy efficiency.

Comparison of Memory Technologies:

Memory	Speed	Capacity	Power Consumption	Main Use
NAND Flash	Slow	Very high	Low	Permanent model storage
DRAM	Very high	High	Moderate	Active AI model and inference data
RRAM	Very high	Medium	Low	Fast, low-power model storage/in-memory computing

Recommended Hierarchical Structure

AI Accelerator/CPU → DRAM → RRAM → NAND Flash

1. **DRAM – Primary Working Memory**
    - Store the currently running AI model and intermediate inference data.
    - Provides high bandwidth and low latency.
    - Use LPDDR5/LPDDR5X for better power efficiency.
2. **RRAM – Fast Model Storage**
    - Store frequently used AI models or model parameters.
    - RRAM is non-volatile, so data remains available without power.
    - It can reduce the need to repeatedly load models from slower NAND Flash.
3. **NAND Flash – Large Storage**
    - Store large AI models, operating system files, datasets, and applications.
    - Provides high capacity at relatively low cost.
    - Models are transferred from NAND to faster memory when required.

Optimized Operation

NAND Flash → RRAM → DRAM → AI Accelerator

- NAND Flash: Stores all models permanently.

- RRAM: Keeps frequently used models close to the processor.
- DRAM: Holds the active model during inference.
- AI Accelerator: Performs computation using data from DRAM.

Recommendation

The best design is a three-level hierarchy using NAND Flash + RRAM + low-power DRAM. NAND Flash provides large-capacity storage, RRAM enables fast and energy-efficient model access, and DRAM provides high-bandwidth working memory.

Result:

This structure provides faster AI model loading, lower power consumption, reduced latency, and efficient edge-AI inference.

5. HIGH-PERFORMANCE GAMING CONSOLE

Problem:

A gaming console suffers from frame drops and stuttering when loading large game scenes because the CPU/GPU cannot receive textures, models, and other game data fast enough.

L3 Cache vs DRAM

Feature	**L3 Cache**	**DRAM**
Speed	Very high	High
Capacity	Small	Large
Latency	Very low	Higher
Bandwidth	High	Very high
Purpose	Frequently used data	Large game assets
Best use	CPU/AI calculations	Textures, models, frame buffers

L3 Cache:

L3 cache keeps frequently accessed CPU data close to the processor. It reduces CPU memory latency but has limited capacity, so it cannot hold large game scenes.

DRAM:

DRAM provides much larger capacity and is essential for storing textures, 3D models, shaders, and other game assets. However, accessing DRAM takes longer than accessing cache.

Proposed Memory Hierarchy:

CPU → L1 → L2 → L3 Cache → High-Bandwidth DRAM → SSD

1. **Optimize L3 Cache**
    - Increase L3 capacity where practical.
    - Keep frequently reused game-engine data, physics data, and AI information in cache.
    - Improve cache prefetching to predict upcoming data.
2. **Use High-Bandwidth DRAM**
    - Use GDDR6/GDDR6X or other high-bandwidth memory technologies suitable for the console architecture.
    - Provide sufficient memory bandwidth for GPU texture and frame-buffer operations.
    - Use multiple memory channels to increase parallel data transfers.
3. **Improve SSD-to-DRAM Streaming**
    - Use a high-speed NVMe/PCIe SSD.
    - Stream assets into DRAM before they are needed.
    - Use data compression to reduce the amount of data transferred.
4. **Use GPU-Optimized Memory Access**
    - Allow the GPU to access required assets efficiently.
    - Keep frequently reused textures and buffers readily available.

Expected Result

The improved hierarchy reduces the time required to load large scenes and prevents the CPU/GPU from waiting for data.

Result:

L3 cache reduces processing latency, while high-bandwidth DRAM handles large game assets. Combining efficient caching, high-bandwidth memory, and fast SSD streaming can reduce frame drops, improve loading performance, and provide smoother gameplay.